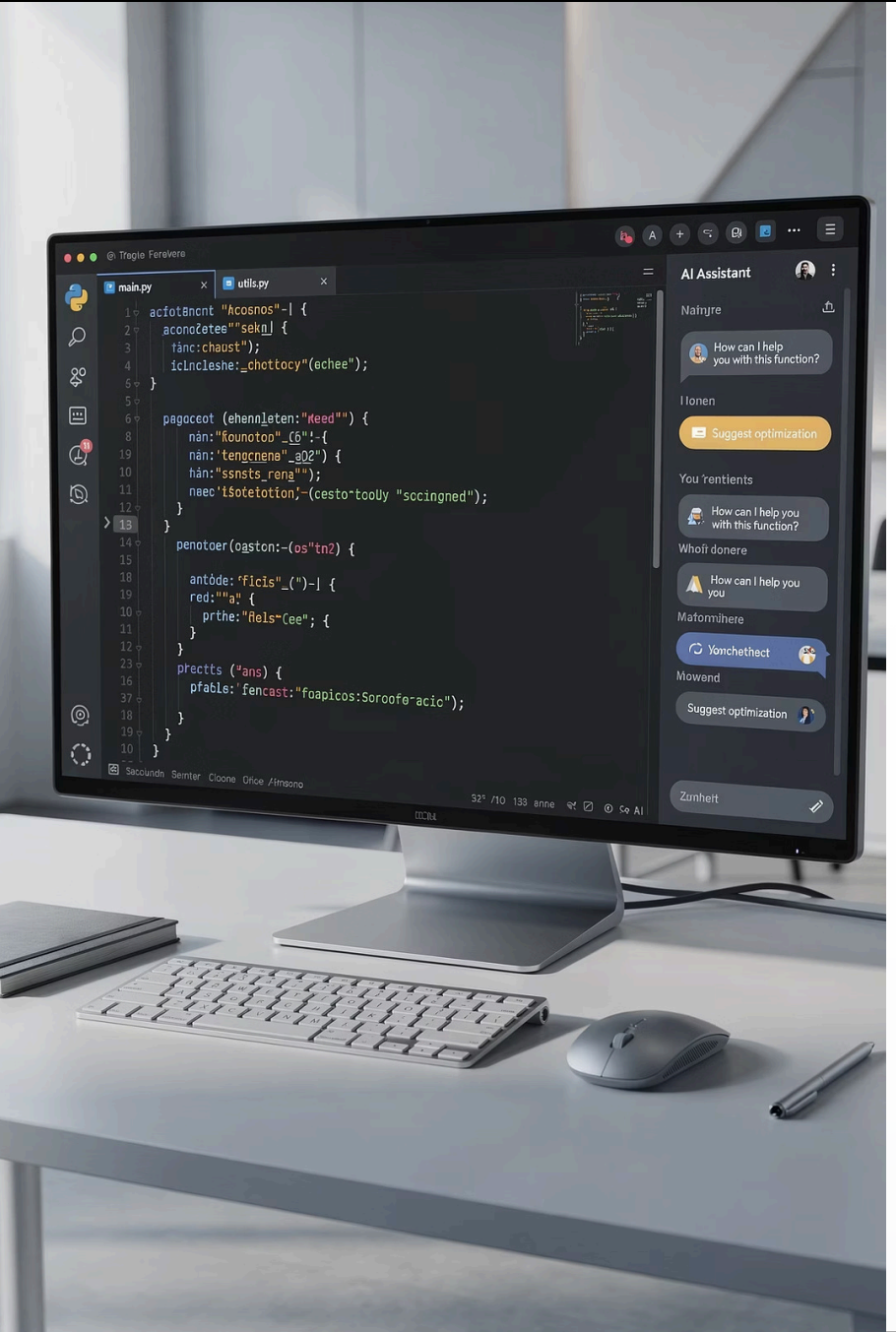
LangChain Beginner's Tutorial

A step-by-step guide to understanding LangChain by building an **Email Humanizer agent** — from your first LLM call to a fully working AI-powered tool.

Table of Contents



CHAPTER 1

What is LangChain?

LangChain is a **Python framework** for building applications powered by Large Language Models (LLMs) like OpenAI's GPT. It handles the plumbing so you focus on the logic.

Without LangChain

Write raw API calls to OpenAI, manually manage prompts, parse responses, and handle tool calling yourself.

With LangChain

Use pre-built components that snap together like building blocks:

`[Your App] → [LangChain] → [OpenAI API] → [GPT Model]`

CHAPTER 2

Core Concepts

Every LangChain app follows this basic pipeline: `[Input] → [Prompt Template] → [LLM] → [Output]`

LLM

The AI model that generates text. *Analogy: The brain.*

Prompt Template

A reusable template with placeholders. *Analogy: A form with blanks.*

Tools

Functions the LLM can call. *Analogy: The hands.*

Agent

An LLM that decides which tools to use. *Analogy: An employee with tools.*

Messages

The conversation format: Human, AI, System. *Analogy: A chat thread.*

Chain vs. Agent

Feature	Chain	Agent
Flow	Fixed: A → B → C	Dynamic: LLM decides the path
Tools	Not used	LLM picks and calls tools
Flexibility	Does the same thing every time	Can adapt based on input
Use Case	Simple, predictable tasks	Complex tasks needing decisions

Start with a Chain to test your logic, then upgrade to an Agent when you need dynamic decision-making.

CHAPTER 3

Setting Up Your Environment

1

Install Dependencies

`pip install langchain langchain-openai openai python-dotenv`

2

Get an OpenAI API Key

Visit `platform.openai.com/api-keys`, create a new key, and copy it.

3

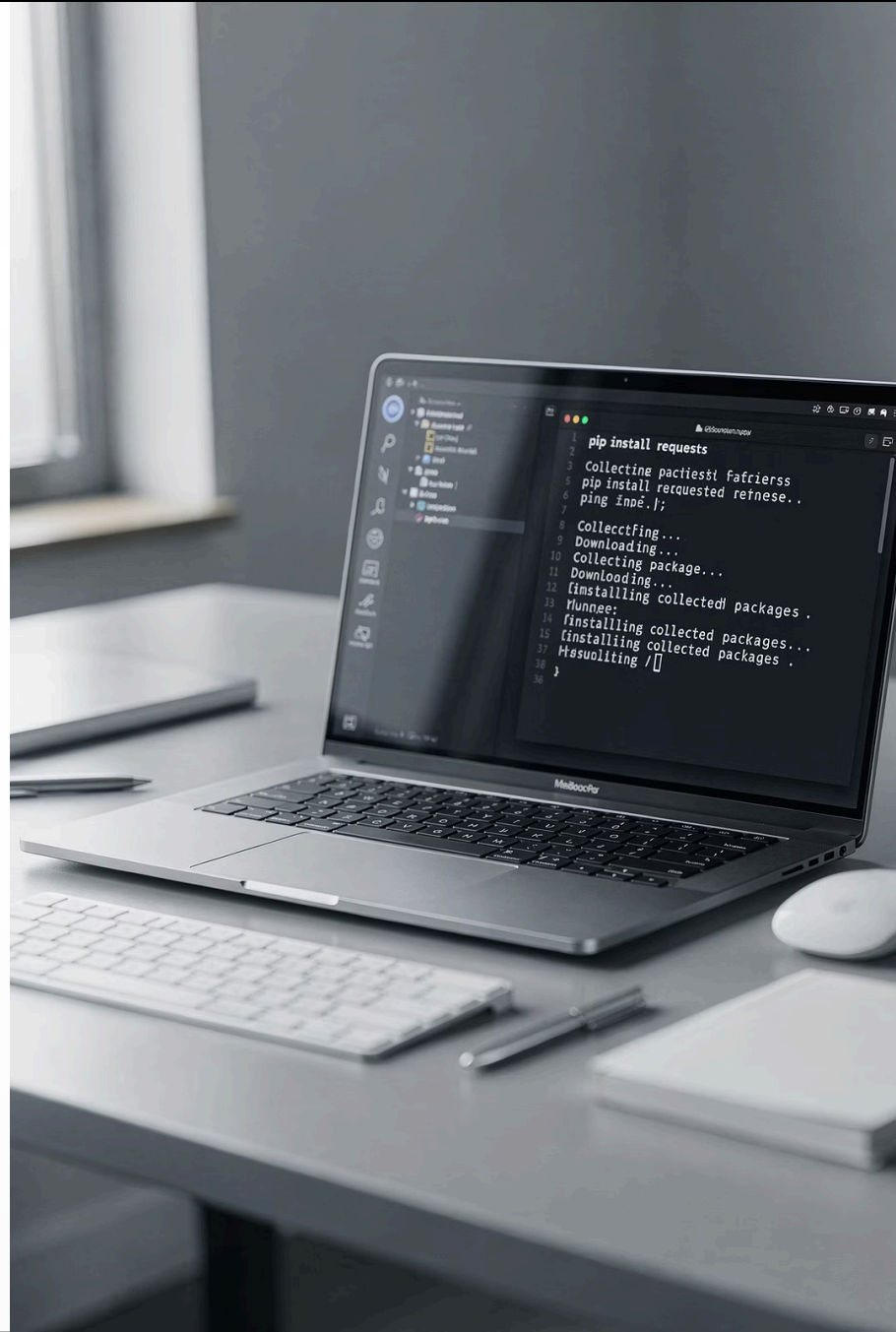
Create a .env File

```
OPENAI_API_KEY=sk-your-actual-key-here
```

4

Load the Key in Python

`load_dotenv()` then `os.getenv("OPENAI_API_KEY")` — keeps your secret key out of your code and git history.



CHAPTER 4

Your First LLM Call

The simplest thing you can do with LangChain — send a message to GPT and get a response.

```
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-4.1-mini", temperature=0.7)

response = llm.invoke("What is LangChain in one sentence?")
print(response.content)
```

ChatOpenAI(...)

Creates a connection to OpenAI's API.

llm.invoke(...)

Sends your text to GPT and waits for a response.

response.content

The actual text GPT returned.

temperature

0 = deterministic, 1 = creative. Use 0.7 for natural writing.

CHAPTER 5

Prompt Templates

Instead of hardcoding prompts, use templates with placeholders for reusable, cleaner code.

```
from langchain_core.prompts import PromptTemplate

template = PromptTemplate(
    input_variables=["topic"],
    template="Write a short paragraph about {topic}."
)

filled_prompt = template.format(topic="artificial intelligence")
response = llm.invoke(filled_prompt)
```

Why Use Templates?

- **Reusable** — same template, different inputs
- **Cleaner code** — separates prompt logic from app logic
- **Easier to test** and modify

Multiple Placeholders

```
PromptTemplate(
 input_variables=[
 "sender","recipient","topic"
 ],
 template="Write an email from
 {sender} to {recipient}
 about {topic}."
)
```

CHAPTER 6

Tools — Giving the LLM Superpowers

A tool is a Python function the agent can call. The `@tool` decorator registers it with LangChain. The **docstring IS the tool description** — the agent reads it to decide when to use the tool.

❌ Bad Docstring

```
@tool
def process(text: str) -> str:
    """Process the text."""
```

Too vague — the agent won't know when to use this.

✅ Good Docstring

```
@tool
def draft_email(idea: str) -> str:
    """Creates a structured email
    draft from a brief idea. Use
    this tool FIRST when the user
    provides an email idea."""
```

Specific — the agent knows exactly when and how to use it.

Always use **type hints** on parameters — LangChain uses these to validate inputs. Vague descriptions confuse the agent.

CHAPTER 7

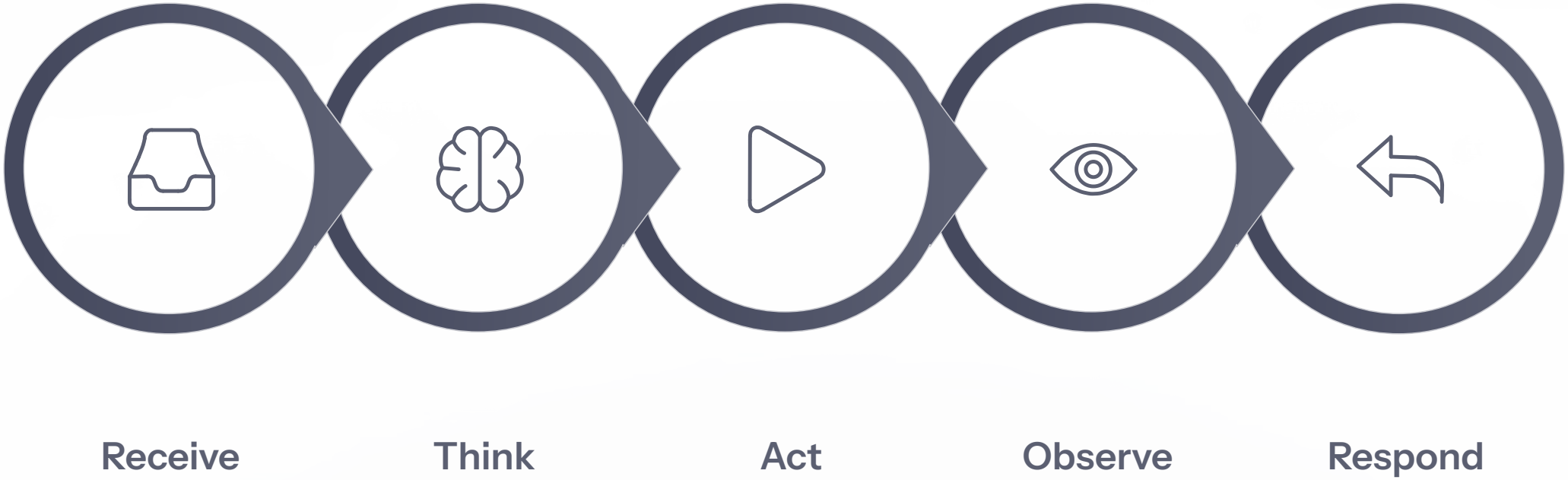
Agents — LLMs That Think and Act

An agent is an LLM that decides which tools to use and in what order. Three ingredients are required:

model
The LLM (GPT) that does the thinking.

tools
The functions the agent can call.

system_prompt
Instructions defining the agent's role and behavior.



Steps 2–4 repeat until the LLM decides it's done, then it returns a final text answer.

The System Prompt — The Agent's Job Description

The system prompt is critical. It tells the agent what role it plays, which tools to use and in what order, and any rules or constraints.

```
SYSTEM_PROMPT = """You are an Email Humanizer assistant.
When the user gives you an email idea:
1. First, use the draft_email tool to create a draft.
2. Then, use the humanize_email tool to make it natural.
3. Return the final email."""
```

A well-written system prompt is the difference between an agent that works reliably and one that behaves unpredictably. Be explicit about tool order and rules.

CHAPTER 8

Putting It All Together — Email Humanizer

Here's the full flow when a user says: *"thank my team for finishing the project on time"*

1

Agent Receives Message

User's email idea enters the agent.

2

Calls draft_email Tool

Creates a PromptTemplate, fills in the idea, sends to LLM → returns a formal email draft.

3

Calls humanize_email Tool

Creates a PromptTemplate for humanization, fills in the draft, sends to LLM → returns a warm, natural email.

4

Agent Responds

Decides it's done and returns the final humanized email to the user.

CHAPTER 9

How the Agent Runs Step by Step

When you run `python email_humanizer.py`, here's what happens in order:

#	What Happens	Code
1	Logging is set up	`logging.basicConfig(...)`
2	API key loaded from .env	`load_dotenv()` + `os.getenv(...)`
3	LLM connection created	`ChatOpenAI(model="gpt-4.1-mini")`
4	Two tools defined	`@tool def draft_email(...)` and `@tool def humanize_email(...)`
5	Agent created	`create_agent(model=llm, tools=tools, ...)`
6	Interactive loop starts	`input("Your email idea: ")`
7	User enters an idea	e.g., "apologize for missing a meeting"
8	Agent runs tool-calling loop	`agent_graph.invoke({"messages": [...]})`
9	Final email is printed	`print(humanized_email)`

CHAPTER 10

Common Mistakes & Tips

❌ Missing API Key

Error: `OPENAI_API_KEY not set`

Fix: Create a `.env` file with your key. Ensure `load_dotenv()` is called before `os.getenv()`.

❌ Bad Tool Docstrings

If the agent doesn't use tools correctly, check the docstring. The agent reads it to decide when and how to use the tool.

❌ Missing Type Hints

Always annotate tool parameters: `def my_tool(text: str) -> str`. LangChain uses these to validate inputs.

❌ Wrong Temperature

`0` = deterministic (factual tasks). `0.7` = some creativity (writing). `1.0` = very random (usually too unpredictable).

✅ Use Logging

Add `logger.info(...)` calls and set `debug=True` on `create_agent` to see the full reasoning.

✅ Start Simple

Build a Chain first (no agent), test it works, then upgrade to an Agent with tools.

CHAPTER 10

Next Steps

Once you're comfortable with this project, try these extensions:

Add More Tools

Build a `translate_email` or `summarize_email` tool to expand the agent's capabilities.

Try Different Models

Swap `gpt-4.1-mini` for `gpt-4o` and compare quality and cost.

Add Memory

Let the agent remember previous emails in the conversation for context-aware responses.

Build a Web UI

Use **Streamlit** or **Gradio** to make the Email Humanizer interactive in a browser.

Explore LangSmith

LangChain's tracing tool to debug and monitor agent runs in production.

LangChain Docs
`docs.langchain.com`

OpenAI API Docs
`platform.openai.com/docs`

LangChain GitHub
`github.com/langchain-ai/langchain`